RNN, LSTM, and GRU — Complete Guide

What is RNN?

RNN (Recurrent Neural Network) is a type of neural network designed to work with **sequential data** — data where the ORDER matters and current output depends on **previous inputs!**

Think of it like this:

Reading this sentence word by word:
"The cat sat on the..."
Your brain REMEMBERS previous words!
Uses that memory to predict next word: "mat"!

Regular ANN: sees each word INDEPENDENTLY
RNN: REMEMBERS previous words → better prediction!

Super Simple Analogies

Reading Story Analogy

Reading a mystery novel:
Chapter 1: Introduced detective John
Chapter 5: "He solved the case"
Who is "He"? → John! (from Chapter 1)

Your brain kept MEMORY of previous chapters!
RNN does the same → keeps memory of previous inputs!

Music Analogy

Listening to a song:
Notes: Do Re Mi Fa...
What comes next? Sol!
But only if you HEARD the previous notes!

Without memory: random guess!
With memory (RNN): follows the melody!

Conversation Analogy

Friend: "My cat is sick"
You: "What's wrong with it?"

Friend: "It won't eat"
You: "Take it to the vet"

Without memory: "What is 'it'?"
With memory: "it" = the cat!
RNN maintains CONTEXT across conversation!

Where is Sequential Data Found?

SEQUENTIAL DATA (order matters!):

TEXT:

"I love machine learning" → words in order!

Sentiment depends on all words together!

SPEECH/AUDIO:

Sound waves over time!

Words formed by sequence of sounds!

TIME SERIES:

Stock prices: [100, 102, 98, 105, ...]

Each price depends on previous prices!

VIDEO:

Sequence of image frames!

Action depends on previous frames!

DNA SEQUENCES:

ATGCCGATCG... (biological sequences)

Gene functions from sequence order!

WEATHER:

Temperature: Mon, Tue, Wed...

Today's weather depends on previous days!

MUSIC:

Notes in sequence form melody!

Next note depends on previous notes!

KEY POINT:

Standard ANN and CNN treat each sample INDEPENDENTLY!

They CANNOT capture sequential dependencies!

RNN is specifically designed for sequences!

PART 1 — RNN Fundamentals

Why Regular ANN Fails for Sequences

PROBLEM WITH REGULAR ANN:

Task: Predict next word given sentence so far!

Input: "The cat sat on the"

Output: "mat"

Method 1: Sliding window ANN

Take last 3 words: [sat, on, the]

Predict: mat

PROBLEM:

Window size is FIXED!

Long-range dependencies missed!

"Mary who John likes __ beautiful"

'__' refers to Mary (far back!)

Fixed window might not reach Mary!

Method 2: Full sentence to ANN (one-hot encoded)

Input: 10,000 × vocabulary_size

Each word = 10,000 position vector

Very sparse and HUGE!

WORSE PROBLEM:

Different sequence lengths!

"Cat sat" (2 words) vs "The big cat sat on mat" (6 words)

ANN needs FIXED input size!

Cannot handle variable length sequences!

RNN SOLUTION:

Process sequence ONE ELEMENT AT A TIME!

Maintain a HIDDEN STATE (memory)!

Hidden state carries information from past!

Works for ANY sequence length!

RNN Structure

RNN CELL (the core unit):

At each time step t:

Two INPUTS:

1. Current input: x_t (e.g., current word)

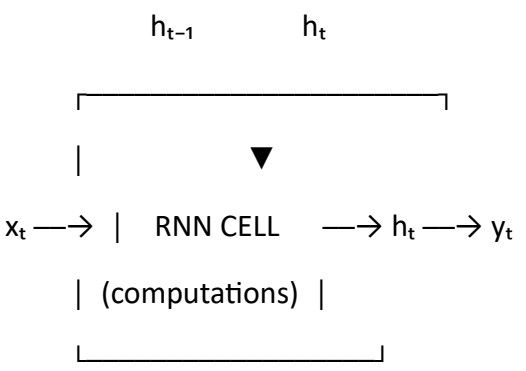
2. Previous hidden state: h_{t-1} (memory from past!)

ONE OUTPUT:

→ Current hidden state: h_t (updated memory!)

→ Output y_t (prediction at this step, if needed)

DIAGRAM:



UNROLLED THROUGH TIME:

h_0 h_1 h_2 h_3

↓ ↓ ↓ ↓

x_1 → [RNN] → x_2 → [RNN] → x_3 → [RNN] → output

↑ ↑ ↑

SAME weights at every step!

Parameter SHARING across time!

RNN Mathematics

Forward Pass Equations

At each time step t:

STEP 1: Compute new hidden state

$h_t = f(W_{xx} \cdot x_t + W_{hh} \cdot h_{t-1} + b_h)$

Where:

x_t = input at time t (dimension: input_size)

h_t = hidden state at time t (dimension: hidden_size)

h_{t-1} = previous hidden state

W_{xx} = input-to-hidden weights (hidden_size × input_size)

W_{hh} = hidden-to-hidden weights (hidden_size × hidden_size)

b_h = hidden bias (hidden_size)

f = activation function (usually tanh!)

STEP 2: Compute output (if needed)

$y_t = g(W_{hy} \cdot h_t + b_y)$

Where:

W_{hy} = hidden-to-output weights (output_size × hidden_size)

b_y = output bias (output_size)

g = output activation (softmax for classification, linear for regression)

INITIAL HIDDEN STATE:

h_o = vector of zeros (no memory yet!)

h_o = [0, 0, 0, ..., 0] (hidden_size zeros)

PARAMETERS (same at every time step!):

W_{xx}, W_{hh}, b_h, W_{hy}, b_y

Total: hidden_size×input_size + hidden_size×hidden_size + 2×hidden_size + output_size×hidden_size + output_size

---

Detailed Math Example

Simple RNN:

input_size = 2 (2 features per step)

hidden_size = 3 (3 hidden units)

output_size = 1 (predict one value)

Initialize (randomly):

W_{xx} = [[0.1, 0.2], (3×2 matrix)

[0.3, 0.4],

[0.5, 0.6]]

W_{hh} = [[0.1, 0.2, 0.3], (3×3 matrix)

[0.4, 0.5, 0.6],

[0.7, 0.8, 0.9]]

b_h = [0, 0, 0]

W_{hy} = [[1.0, 1.0, 1.0]] (1×3 matrix)

b_y = [0]

Initial hidden state:

$h_o = [0, 0, 0]$

TIME STEP t=1:

Input $x_1 = [1.0, 0.5]$

Compute h_t:

$W_{xx} \cdot x_1 = [[0.1, 0.2], \times [1.0] = [0.1\times1+0.2\times0.5] = [0.2]$

$[0.3, 0.4], \quad [0.5] \quad [0.3\times1+0.4\times0.5] \quad [0.5]$

$[0.5, 0.6]] \qquad [0.5\times1+0.6\times0.5] \quad [0.8]$

$W_{hh} \cdot h_o = W_{hh} \times [0,0,0] = [0, 0, 0]$ (zero at start)

$z_1 = W_{xx} \cdot x_1 + W_{hh} \cdot h_o + b_h$

$= [0.2, 0.5, 0.8] + [0,0,0] + [0,0,0]$

$= [0.2, 0.5, 0.8]$

$h_1 = \tanh(z_1) = [\tanh(0.2), \tanh(0.5), \tanh(0.8)]$

$= [0.197, 0.462, 0.664]$

$y_1 = W_{hy} \cdot h_1 + b_y$

$= [1.0, 1.0, 1.0] \cdot [0.197, 0.462, 0.664] + 0$

$= 0.197 + 0.462 + 0.664$

$= 1.323$

TIME STEP t=2:

Input $x_2 = [0.8, 0.2]$

Previous state $h_1 = [0.197, 0.462, 0.664]$

$W_{xx} \cdot x_2 = [0.1\times0.8+0.2\times0.2,\ 0.3\times0.8+0.4\times0.2,\ 0.5\times0.8+0.6\times0.2]$

$= [0.12, 0.32, 0.52]$

$W_{hh} \cdot h_1 = [3\times3 \text{ matrix}] \times [0.197, 0.462, 0.664]$

$= [0.1\times0.197+0.2\times0.462+0.3\times0.664,\ ...]$

$= [0.0197+0.0924+0.1992, ...]$

$= [0.311, ...]$ (continuing computation...)

$h_2 = \tanh(W_{xx}\cdot x_2 + W_{hh}\cdot h_1 + b_h)$

Now h_2 contains information from BOTH x_1 AND x_2!

Memory from previous step carried forward!

Types of RNN Architectures

Based on input/output patterns:

1. ONE-TO-ONE:

   Input: single item

   Output: single item

   Example: Standard ANN (not really RNN)

2. ONE-TO-MANY:

   Input: single item

   Output: sequence

   Example: Image → describe in words!

   Architecture: x → [RNN] → y_1 → y_2 → y_3...

3. MANY-TO-ONE:

Input: sequence

Output: single item

Example: Sentence → sentiment (positive/negative)

Architecture: $x_1 \rightarrow x_2 \rightarrow x_3 \rightarrow$ [RNN] $\rightarrow y$

4. MANY-TO-MANY (same length):

Input: sequence

Output: sequence (same length!)

Example: Named entity recognition

Architecture: $x_1 \rightarrow y_1$, $x_2 \rightarrow y_2$, $x_3 \rightarrow y_3$

5. MANY-TO-MANY (different length):

Input: sequence

Output: sequence (different length!)

Example: Machine translation (Encoder-Decoder)

Architecture: Encode entire sentence → decode to output

Backpropagation Through Time (BPTT)

BPTT = Backpropagation for RNNs!

Unroll RNN through time, then backpropagate!

FORWARD PASS:

Compute $h_1, h_2, ..., h_t$ sequentially!

Compute outputs and loss at each step!

Total loss:

$L = \Sigma_t L_t$ (sum of losses at each time step)

BACKWARD PASS:

Compute gradients flowing BACKWARDS through time!

For final time step T:

$\partial L/\partial h_t = \partial L_t/\partial h_t$ (direct gradient from output)

For earlier time steps t < T:

$\partial L/\partial h_t = \partial L_t/\partial h_t + W_{hh}^T \times \partial L/\partial h_{t+1}$

Each step carries gradient BACK to previous step!

Through multiplication by W_{hh}^T each time!

PROBLEM:

Long sequences: T = 100 or 1000 steps!

Gradient multiplied by W_{hh} at EACH step!

If $||W_{hh}|| < 1$: gradient VANISHES (shrinks to 0)

If $||W_{hh}|| > 1$: gradient EXPLODES (grows to ∞)

This is the VANISHING GRADIENT PROBLEM!

The MAIN limitation of simple RNNs!

PART 2 — Vanishing Gradient Problem

What is the Vanishing Gradient Problem?

MATHEMATICAL EXPLANATION:

Gradient at time step t:

$\partial L/\partial W = \Sigma_t (\partial L_t/\partial W)$

For gradient to reach time step 1 from T:

Must multiply through chain of gradients:

$\partial h_t/\partial h_1 = \Pi_{k=2}^{T} \partial h_k/\partial h_{k-1}$

Each $\partial h_k/\partial h_{k-1} = W_{hh} \times \text{diag}(f'(z_k))$

For tanh activation:

$f'(z) = 1 - \tanh^2(z) \leq 1$

So each multiplication:

$\leq ||W_{hh}|| \times 1$

If $||W_{hh}|| = 0.9$:

After 10 steps: 0.9^10 = 0.35

After 50 steps: 0.9^50 = 0.005

After 100 steps: 0.9^100 = 0.000027

Gradient essentially ZERO at early time steps!

Early parts of sequence have NO EFFECT on learning!

Model cannot remember long-term dependencies!

Vanishing Gradient Analogy

TELEPHONE GAME ANALOGY:

First person says: "The original message was important"

Each person repeats but with slight distortion!

Message garbled more and more!

Person 100: barely any original information!

Gradients are like the MESSAGE!

Each backward step: message weakens!

After many steps: gradient (message) disappeared!

Early layers (early time steps) get NO gradient!

They cannot learn!

REAL CONSEQUENCE:

"The movie I watched yesterday was great but

the ending was terrible and overall I did not enjoy it"

→ Sentiment: NEGATIVE (due to "did not enjoy")

Simple RNN:

→ Sees "it" as last meaningful word

→ Might say: positive (based on recent "it"?)

→ Cannot connect "did not enjoy" to early "movie"!

→ Vanishing gradient → cannot learn this connection!

Exploding Gradient Problem

OPPOSITE PROBLEM:

If $||W_{hh}|| > 1$:

After 10 steps: (1.1)^10 = 2.59

After 50 steps: (1.1)^50 = 117.4

After 100 steps: (1.1)^100 = 13780!

Gradients EXPLODE to infinity!

Weights update by HUGE amounts!

Training becomes completely unstable!

Loss goes to NaN!

EASIER TO FIX (compared to vanishing):

Gradient Clipping!

If ||gradient|| > threshold:

gradient = gradient × (threshold / ||gradient||)

Simply limit gradient magnitude!

Works well in practice!

But vanishing gradient needs architectural fix!

→ This is why LSTM and GRU were invented!

RNN Limitations Summary

1. VANISHING GRADIENT:

Cannot learn long-term dependencies!

Information from far past disappears!

2. SEQUENTIAL COMPUTATION:

Cannot parallelize! Must process t=1, then t=2, then t=3...

Very slow for long sequences!

3. MEMORY IS OVERWRITTEN:

Hidden state is REPLACED at each step!

Old information can be lost!

4. FIXED HIDDEN SIZE:

Limited memory capacity!

Cannot explicitly control what to remember!

SOLUTIONS:

Vanishing gradient → LSTM or GRU!

Parallelization → Transformers!

Better memory → Attention mechanism!

PART 3 — LSTM (Long Short-Term Memory)

What is LSTM?

LSTM (Long Short-Term Memory) is a special RNN architecture with a **memory cell** and three **gates** that control what information to **keep, forget, and output** — solving the vanishing gradient problem!

Super Simple Analogies

Traveler's Luggage Analogy

Traveler going on a long journey (sequence)!

Regular RNN:
Traveler has ONE small backpack!
At each city: throw everything out, pack new stuff!
By city 100: nothing from city 1 remains!

LSTM:
Traveler has:
→ BIG SUITCASE (cell state): stores important things long-term!
→ SMALL BACKPACK (hidden state): carries current context!

FORGET GATE: "Do I still need my winter jacket?" (decide what to remove from suitcase)
INPUT GATE: "Should I pack this souvenir?" (decide what to add to suitcase)
OUTPUT GATE: "What do I take out of suitcase today?" (decide what to use now)

Traveler can remember things from city 1 when in city 100!

Control Panel Analogy

LSTM is like a control panel with three knobs:

Forget Knob: How much of old memory to keep?
(0 = delete everything, 1 = keep everything)

Input Knob: How much new information to store?
(0 = ignore new info, 1 = store everything)

Output Knob: How much memory to expose now?
(0 = hide memory, 1 = show everything)

Model LEARNS how to set these knobs!
Different for each sequence and each step!

LSTM Key Concepts

LSTM has TWO states (unlike RNN's one!):

1. CELL STATE (C_t): Long-term memory!

   Flows through with minimal modification!

   Like a conveyor belt of information!

   Gradients flow easily (no shrinking!)

2. HIDDEN STATE (h_t): Short-term memory!

   Output at current step!

   Used for predictions!

   Like RNN's hidden state!

THREE GATES:

Gates control information flow!

Each gate outputs values 0 to 1!

0 = block completely

1 = pass completely

0.7 = pass 70%

FORGET GATE (f_t): What to forget from cell state!

INPUT GATE (i_t): What new info to add to cell state!

OUTPUT GATE (o_t): What part of cell state to output!

LSTM Mathematics — Step by Step

Step 1 — Forget Gate

FORGET GATE decides:

"How much of previous cell state to keep?"

$$f_t = \sigma(Wf \cdot [h_{t-1}, x_t] + bf)$$

Where:

$[h_{t-1}, x_t]$ = concatenation of previous hidden state and current input!

Wf = forget gate weights (learned!)

bf = forget gate bias

σ = sigmoid function (output 0 to 1!)

SIGMOID OUTPUT:

f_t close to 1: REMEMBER this information!

f_t close to 0: FORGET this information!

f_t = 0.8: keep 80% of this memory!

EXAMPLE:

Reading: "Mary went to the store. She bought..."

When we see "She":

→ Forget gate might set high values for "Mary"

→ Keeps "Mary" in cell state

→ So we know "She" = Mary!

When new topic starts: "The cat sat..."

→ Forget gate reduces memory of "Mary"

→ She's no longer relevant to this new sentence!

DIMENSION:

Wf is matrix of size: hidden_size × (hidden_size + input_size)

f_t has dimension: hidden_size (one value per hidden unit)

Step 2 — Input Gate and Candidate

INPUT GATE decides:

"What new information to store in cell state?"

TWO PARTS:

Part A: Input Gate (decides WHICH to update)

$i_t = \sigma(Wi \cdot [h_{t-1}, x_t] + bi)$

Output: 0 to 1 (which memory cells to update)

Part B: Candidate Values (WHAT to add)

$\tilde{C}_t = \tanh(Wc \cdot [h_{t-1}, x_t] + bc)$

Output: -1 to 1 (new candidate memory values)

WHY TWO PARTS?

i_t (gate): which cells to update (0 or 1 decision!)

$\tilde{C}_t$ (candidate): what values to write (-1 to 1 range!)

Combined: only update selected cells with new values!

EXAMPLE:

"Mary went to the store. John went to..."

After seeing "John":

→ Input gate: update the "subject" memory cell!

(i_t high for "subject" cell)

→ Candidate: $\tilde{C}_t$ encodes "John"

→ Cell updated: subject = John!

Previous subject was Mary

Now updated to John!

Step 3 — Update Cell State

UPDATE CELL STATE:

$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$

Where $\odot$ = element-wise multiplication!

TERM 1: $f_t \odot C_{t-1}$

→ Old cell state × forget gate

→ FORGET unwanted information!

→ f_t=0: forget this part completely!

→ f_t=1: keep this part completely!

TERM 2: $i_t \odot \tilde{C}_t$

→ Input gate × candidate values

→ ADD new selected information!

→ i_t=0: add nothing!

→ i_t=1: add full candidate values!

EXAMPLE:

C_{t-1} = [0.8, 0.3, -0.5] (previous cell: remembers "Mary")

f_t = [0.9, 0.1, 0.8] (forget most of position 2!)

i_t = [0.2, 0.9, 0.1] (input mostly to position 2!)

$\tilde{C}_t$ = [0.3, 0.7, -0.2] (new candidate values)

$f_t \odot C_{t-1}$ = [0.72, 0.03, -0.4] (mostly kept except position 2)

$i_t \odot \tilde{C}_t$ = [0.06, 0.63, -0.02] (mostly added to position 2)

C_t = [0.78, 0.66, -0.42]

↑ ↑

mostly old mostly new

(Mary kept) (John added)

Cell state updated with new subject (John)!

But maintained other memories (Mary mostly forgotten)!

Step 4 — Output Gate

OUTPUT GATE decides:

"What part of cell state to expose as hidden state?"

$o_t = \sigma(Wo \cdot [h_{t-1}, x_t] + bo)$

$h_t = o_t \odot \tanh(C_t)$

WHERE:

$\tanh(C_t)$ = cell state squeezed to [-1, 1]

o_t = gate controlling how much to expose

INTERPRETATION:

Cell state may contain lots of information

Not all relevant for current output!

Output gate selects relevant portion!

EXAMPLE:

Cell state contains: [subject=John, location=store, tense=past]

Current task: predict next word after "John went to the..."

Output gate: high for location, lower for subject

→ Expose: location information more!

→ h_t highlights: "store" information!

→ Next word prediction: "store" (makes sense!)

FINAL OUTPUT:

h_t = output (used for predictions!)

Also fed to NEXT LSTM cell as h_{t-1}!

Complete LSTM Forward Pass Summary

FOUR EQUATIONS (run at each time step):

Forget gate: $f_t = \sigma(Wf \cdot [h_{t-1}, x_t] + bf)$

Input gate: $i_t = \sigma(Wi \cdot [h_{t-1}, x_t] + bi)$

Candidate: $\tilde{C}_t = \tanh(Wc \cdot [h_{t-1}, x_t] + bc)$

Output gate: $o_t = \sigma(Wo \cdot [h_{t-1}, x_t] + bo)$

Cell update: $C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$

Hidden update: $h_t = o_t \odot \tanh(C_t)$

PARAMETERS:

Wf, Wi, Wc, Wo = weight matrices (each: hidden × (hidden+input))

bf, bi, bc, bo = bias vectors (each: hidden)

Total parameters per LSTM cell:

4 × [hidden_size × (hidden_size + input_size) + hidden_size]

= 4 × hidden_size × (hidden_size + input_size + 1)

For hidden=128, input=64:

= 4 × 128 × (128 + 64 + 1) = 4 × 128 × 193 = 98,816 parameters!

More parameters than simple RNN

But MUCH better at learning long-term dependencies!

Complete LSTM Example — Sentiment Analysis

Task: Predict sentiment of movie review

Input: "This film was absolutely terrible"

Output: Negative sentiment

Vocabulary: {This:0, film:1, was:2, absolutely:3, terrible:4}

Word embeddings: each word → 10-dim vector

LSTM hidden size: 5

PROCESSING:

t=1: x_1 = embed("This")

h_0 = [0,0,0,0,0], C_0 = [0,0,0,0,0]

$f_1 = \sigma(W_f \cdot [h_0, x_1] + b_f)$ → forget previous context

$i_1 = \sigma(W_i \cdot [h_0, x_1] + b_i)$ → what to store

$\tilde{C}_1 = \tanh(W_c \cdot [h_0, x_1] + b_c)$ → candidate

$o_1 = \sigma(W_o \cdot [h_0, x_1] + b_o)$ → what to output

$C_1 = f_1 \odot C_0 + i_1 \odot \tilde{C}_1$ (mostly new since C_0=0)

$h_1 = o_1 \odot \tanh(C_1)$

(h_1 encodes: "neutral start, review begins")

t=2: x_2 = embed("film")

Uses h_1 and C_1 from previous step!

h_2 encodes: "this is about a film"

t=3: x_3 = embed("was")

h_3 encodes: "this film was (in past tense)"

t=4: x_4 = embed("absolutely")

IMPORTANT! "Absolutely" is intensifier!

LSTM should remember this modifies what comes next!

Input gate: HIGH → store "absolutely" strongly!

h_4, C_4 encode: "very strong modifier coming"

t=5: x_5 = embed("terrible")

"Terrible" = negative word!

With "absolutely" in memory (from C_4):

LSTM: "absolutely terrible" = VERY negative!

h_5 encodes: very negative sentiment!

FINAL PREDICTION:

Use h_5 (last hidden state)!

Linear layer: h_5 → 2 outputs (positive/negative)

Softmax: [0.03, 0.97]

Prediction: NEGATIVE (97% confidence)!

Without LSTM memory:

"terrible" alone = negative (80%)

"absolutely terrible" (with memory) = very negative (97%)!

LSTM used long-range dependency!

Why LSTM Solves Vanishing Gradient

MATHEMATICAL REASON:

Cell state update:

$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$

Gradient through cell state:

$\partial C_t / \partial C_{t-1} = f_t$ (just the forget gate values!)

If $f_t \approx 1$ (keep everything):

Gradient ≈ 1 at each step!

No vanishing!

COMPARE TO SIMPLE RNN:

$\partial h_t / \partial h_{t-1} = W_{hh} \times f'(z_t)$

Both W_{hh} and $f'(z_t) < 1$ → vanishing!

LSTM cell state:

Multiplication by f_t (learned!)

Network can learn $f_t \approx 1$ when needed!

Gradient flows with minimal decay!

This is why LSTM CAN learn long-range dependencies!

Cell state = gradient highway!

Information flows across many steps!

VISUAL:

Simple RNN gradient:

Step T → T-1 → T-2 → ... → 1

Gradient: 1 → 0.5 → 0.25 → ... → ~0 (vanished!)

LSTM gradient (through cell state):

Step T → T-1 → T-2 → ... → 1

Gradient: 1 → 0.95 → 0.90 → ... → 0.6 (still alive!)

PART 4 — GRU (Gated Recurrent Unit)

What is GRU?

GRU (Gated Recurrent Unit) is a **simplified version of LSTM** with only **TWO gates** instead of three — faster to train, often performs similarly to LSTM!

GRU Analogy

LIGHT TRAVELER ANALOGY:

LSTM = traveler with big suitcase + backpack + sorting system

Lots of luggage but can carry everything!

Slower to pack/unpack!

GRU = experienced traveler who travels LIGHT!

Only brings what's needed right now!

One smart bag instead of multiple!

Faster to travel! Often just as effective!

GRU simplified LSTM by:

Merging cell state and hidden state into ONE!

Merging forget and input gates into ONE!

Result: Fewer parameters, faster training!

Performance: Often same as LSTM (sometimes better!)

GRU Mathematics

GRU Gates

GRU has only TWO gates:

1. UPDATE GATE (z_t): Combined forget + input gate!

Controls how much past to keep AND how much new to add!

2. RESET GATE (r_t): Controls how much past to consider

when creating new candidate!

NO SEPARATE CELL STATE!

Only hidden state h_t (like simple RNN)

But controlled by smart gates!

GRU Equations

STEP 1: Update Gate

$z_t = \sigma(Wz \cdot [h_{t-1}, x_t] + bz)$

Output: 0 to 1

$z_t \approx 1$: Keep mostly OLD hidden state!

$z_t \approx 0$: Mostly use NEW candidate!

STEP 2: Reset Gate

$r_t = \sigma(Wr \cdot [h_{t-1}, x_t] + br)$

Output: 0 to 1

$r_t \approx 1$: Keep full previous state for candidate calculation!

$r_t \approx 0$: Ignore previous state for candidate!

(Reset memory → candidate based more on current input)

STEP 3: Candidate Hidden State

$\tilde{h}_t = \tanh(W_h \cdot [r_t \odot h_{t-1}, x_t] + b_h)$

Note: $r_t \odot h_{t-1}$ = previous hidden state masked by reset gate!

If $r_t \approx 0$: candidate ignores past → based on current input!

If $r_t \approx 1$: candidate uses full past information!

STEP 4: Update Hidden State

$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$

BEAUTIFUL! Update gate controls interpolation:

$z_t = 0$: $h_t = h_{t-1}$ (completely keep old state!)

$z_t = 1$: $h_t = \tilde{h}_t$ (completely use new candidate!)

$z_t = 0.7$: 70% new, 30% old! (weighted combination!)

Compare to LSTM:

LSTM: separate forget gate (f_t) and input gate (i_t)

GRU: single update gate (z_t) handles both!

Simpler but effective!

GRU Example

Task: Predict if review is positive/negative

Input: "Not good at all"

KEY: "Not" negates "good"!

Model must remember "Not" when seeing "good"!

t=1: x_1 = embed("Not")

$h_0 = [0,0,0,0]$

Reset gate $r_1 = \sigma(...)$ (based on h_o and x_1)

Update gate $z_1 = \sigma(...)$ (based on h_o and x_1)

$\tilde{h}_1 = \tanh(W_h \cdot [r_1 \odot h_o, x_1])$

(encodes: "negation incoming!")

$h_1 = (1-z_1) \odot h_o + z_1 \odot \tilde{h}_1$

h_1 encodes: "Not" (negation word!)

t=2: x_2 = embed("good")

Reset gate r_2: LOW!

→ Don't reset previous memory!

→ "Not" from h_1 should influence candidate!

$\tilde{h}_2 = \tanh(W_h \cdot [r_2 \odot h_1, x_2])$

Since r_2 is low: h_1 (negation) heavily influences candidate!

Candidate: "NOT good" → negative signal!

Update gate z_2: HIGH

→ Update hidden state significantly with new candidate!

h_2 encodes: "negated good" = negative sentiment!

t=3: x_3 = embed("at")

t=4: x_4 = embed("all")

"At all" reinforces the negation!

LSTM/GRU should encode: very negative!

Final h_4 → classifier → NEGATIVE!

GRU correctly handled negation dependency!

LSTM vs GRU Comparison

STRUCTURE:

LSTM: Cell state + Hidden state (two states!)

GRU: Only Hidden state (one state!)

GATES:

LSTM: 3 gates (forget, input, output)

GRU: 2 gates (update, reset)

PARAMETERS:

LSTM: 4 × hidden_size × (hidden_size + input_size + 1)

GRU: 3 × hidden_size × (hidden_size + input_size + 1)

For hidden=128, input=64:

LSTM: 98,816 parameters

GRU: 74,112 parameters (25% fewer!)

TRAINING SPEED:

GRU: Faster (fewer parameters, fewer computations!)

LSTM: Slower but potentially more expressive!

PERFORMANCE:

Many tasks: Similar performance!

Very long sequences: LSTM sometimes better!

Smaller datasets: GRU sometimes better!

WHEN TO USE:

LSTM:

→ Very long sequences (100s-1000s steps)

→ Need precise control of memory

→ Enough data to learn more parameters

→ Speech recognition, language modeling

GRU:

→ Medium sequences

→ Faster training needed

→ Limited training data

→ Sentiment analysis, simpler tasks

→ When to experiment first!

RULE OF THUMB:

Try GRU first (faster!)

If performance not enough: switch to LSTM!

LSTM vs GRU Side-by-Side

Feature	**Simple RNN**	**LSTM**	**GRU**
States	1 (hidden)	2 (cell + hidden)	1 (hidden)
Gates	0	3	2
Parameters	Fewest	Most	Medium
Long-term memory	Poor	Excellent	Good

Training speed	Fast	Slow	Medium
Vanishing gradient	Severe	Solved	Solved
Best for	Very short	Long sequences	Most tasks

PART 5 — Advanced RNN Concepts

Bidirectional RNN

PROBLEM WITH STANDARD RNN:

Only looks at PAST context!

"The ___ is in the sky" → predict "star"

Need both past AND future context sometimes!

"The ___ is in the sky and it's bright"

"bright" comes AFTER blank → helps predict!

"clouds are blocking the ___" → moon? star? sun?

Both past and future help!

SOLUTION: Bidirectional RNN!

FORWARD RNN: reads left → right (past context)

BACKWARD RNN: reads right → left (future context)

For each position t:

→ Forward RNN gives: $h\rightarrow t$ (uses past: $x_1,...,x_t$)

→ Backward RNN gives: $h\leftarrow t$ (uses future: $x_t,...,x_n$)

COMBINE: $h_t = [h\rightarrow t, h\leftarrow t]$ (concatenate!)

PREDICTION:

$y_t = f(W_y \cdot [h\rightarrow t, h\leftarrow t] + b_y)$

Uses BOTH past AND future context!

EXAMPLE — Named Entity Recognition:

"Obama was the president of USA"

Standard RNN sees "Obama" → not sure yet

Bidirectional: also sees "president" → "Obama is a person!"

Future context helps understand current word!

WHEN TO USE:

Text classification (see whole sentence)

Named entity recognition

Machine translation (encoder)

Question answering

WHEN NOT TO USE:

Real-time speech (can't see future)

Language modeling (predict next word → future cheating!)

Time series prediction (can't see future)

Deep (Stacked) RNN

STACKED RNN:

Multiple RNN layers on top of each other!

Layer 1: processes raw input

Output h_{1t} → input to Layer 2

Layer 2: processes Layer 1's output

Output h_{2t} → input to Layer 3

Layer 3: processes Layer 2's output

Output h_{3t} → final hidden state!

DIAGRAM:

h_{3t} → final output

↑

[RNN Layer 3] ← h_{2t-1}

↑

[RNN Layer 2] ← h_{1t-1}

↑

[RNN Layer 1] ← x_t

WHY STACKED?

Layer 1: learns low-level patterns (words, sounds)

Layer 2: learns mid-level patterns (phrases, tone)

Layer 3: learns high-level patterns (meaning, sentiment)

Each layer adds ABSTRACTION!

Like CNN layers for images!

TYPICAL DEPTH:

2-4 layers for most tasks!

More can overfit with small data!

Add dropout between layers!

PERFORMANCE:

Usually better than single layer!

With LSTM: most commonly 2-3 stacked LSTM layers!

Encoder-Decoder (Seq2Seq)

PROBLEM: Input and output sequences DIFFERENT lengths!

Example — Machine Translation:

English: "I love you" (3 words)

Hindi: "Main tumse pyar karta hoon" (5 words)

Can't use simple many-to-many (same length!)

Need different length input and output!

SOLUTION: Encoder-Decoder Architecture!

ENCODER:

Reads entire input sequence!

Produces CONTEXT VECTOR (fixed-size summary!)

English: I → love → you

LSTM: $h_1 \rightarrow h_2 \rightarrow h_3$ = CONTEXT VECTOR

Context vector = compressed representation of input!

DECODER:

Takes context vector!

Generates output sequence one token at a time!

Context → Main → tumse → pyar → karta → hoon

DECODER generates until it produces <EOS> (end of sequence)!

PROBLEM:

Single context vector bottleneck!

Long sentences → context vector can't store everything!

Information loss for long sequences!

SOLUTION: ATTENTION MECHANISM!

Decoder can look at ALL encoder hidden states!

Not just final context vector!

Learns WHICH input positions to attend to!

Foundation of Transformers!

PART 6 — Training Considerations

Practical Training Tips

Gradient Clipping

WHY:

Exploding gradients in RNNs → NaN loss!

HOW:

If ||gradient|| > threshold:

gradient = gradient × (threshold/||gradient||)

Common threshold: 1.0 or 5.0

WHEN TO USE:

Always with RNNs! (especially LSTM/GRU)

Monitor: if loss suddenly becomes NaN → gradient exploding!

Solution: reduce learning rate OR add gradient clipping!

Sequence Padding

PROBLEM:

Sequences in a batch have different lengths!

"I love it" (3 words)

"This is absolutely terrible" (4 words)

"Great" (1 word)

Can't stack into matrix (different sizes)!

SOLUTION: PADDING!

Add special <PAD> token to shorter sequences!

After padding to length 4:

"I love it <PAD>" → [idx, idx, idx, 0]

"This is absolutely terrible" → [idx, idx, idx, idx]

"Great <PAD> <PAD> <PAD>" → [idx, 0, 0, 0]

Now all same length → can batch!

MASKING:

Tell model to IGNORE padding positions!

Loss not computed for <PAD> positions!

Gradients not computed for <PAD>!

Only actual words contribute to learning!

Dropout in RNN

STANDARD DROPOUT IN RNN:

Apply between STACKED RNN layers (vertical)!

NOT between time steps (horizontal)!

Why? Applying dropout along time corrupts sequential information!

PROPER DROPOUT PLACEMENT:

Input → Dropout → LSTM Layer 1 → Dropout → LSTM Layer 2 → Dropout → Output

VARIATIONAL DROPOUT:

Apply SAME dropout mask at EVERY time step!

Mask decided once per sequence!

Consistent regularization!

Better than changing mask each step!

RECURRENT DROPOUT:

Dropout on recurrent connections (W_{hh})!

Applied consistently across time steps!

More effective regularization for RNNs!

TYPICAL DROPOUT RATES:

Input dropout: 0.2-0.3

Output dropout: 0.2-0.5

Recurrent dropout: 0.1-0.3

Learning Rate for RNN

RNNs are MORE SENSITIVE to learning rate!

Gradients can vanish OR explode easily!

RECOMMENDATIONS:

Adam optimizer: lr = 0.001 (start here!)

SGD + gradient clipping: lr = 0.01 or 0.1

LEARNING RATE SCHEDULE:

Start with 0.001

Reduce by 2-10× when validation loss plateaus!

Common: ReduceLROnPlateau!

WARMUP:

Start very small: 1e-6

Gradually increase to 1e-3 over first few epochs

Then follow schedule!

Prevents early instability!

PART 7 — Applications

Real-World Applications

Application 1 — Text Generation

TASK: Generate new text similar to training data!

TRAINING:

Input: "The cat sat on the mat"

At each step: predict next character/word!

Input: "The cat sat on" → Output: "the"

Input: "cat sat on the" → Output: "mat"

INFERENCE (generation):

Start with seed: "The cat"

Predict: "sat" (with probability)

Append: "The cat sat"

Predict: "on"

Append: "The cat sat on"

... continue generating!

ARCHITECTURE:

Character-level LSTM:

Vocabulary: all ASCII characters (~100)

Embedding: one-hot or learned

LSTM: 512 units, 2-3 layers

Output: softmax over vocabulary

TEMPERATURE (controls randomness):

Low temp (0.1): conservative, repetitive!

High temp (1.5): creative, random!

Medium (0.7): balanced creativity!

EXAMPLES:

Shakespeare LSTM trained on Shakespeare:

"To be, or not to be, that is the question..."

Generate: Sounds like Shakespeare!

Code LSTM trained on Python code:

Generate: Valid-looking Python code!

Application 2 — Machine Translation

TASK: Translate English → French

ARCHITECTURE: Encoder-Decoder LSTM!

ENCODER (English):

"I love Paris"

embed → LSTM (bidirectional) → context vector

Or better: encode to all hidden states (for attention)!

DECODER (French):

Start with <SOS> (start of sequence)!

+ context vector

→ Predict: "J'"

Append "J'" → predict: "aime"

Append "aime" → predict: "Paris"

Append "Paris" → predict: <EOS>

Stop generation!

Output: "J'aime Paris"

WITH ATTENTION:

Decoder at each step looks at ALL encoder states!

Attends to most relevant input words!

"J'" attends to "I"

"aime" attends to "love"

"Paris" attends to "Paris"

Much better than single context vector!

Application 3 — Time Series Forecasting

TASK: Predict future stock prices!

INPUT: Past 30 days of prices

OUTPUT: Next day's price

DATA FORMAT:

Day 1-30: $[p_1, p_2, ..., p_{30}]$ → predict p_{31}

Sliding window training:

Window $[p_1,...,p_{30}]$ → target p_{31}

Window $[p_2,...,p_{31}]$ → target p_{32}

...

ARCHITECTURE:

Input: 30 time steps, 1 feature (price)

Or multi-feature: [price, volume, RSI, MACD...]

LSTM: 64-128 units

Output: linear (predict continuous price)

Loss: MSE

MULTI-STEP PREDICTION:

Predict 7 days ahead:

Output: 7 values

Loss: MSE over all 7 predictions

PREPROCESSING (critical!):

Normalize prices to [0,1] or z-score!

RNN sensitive to scale!

Remember to denormalize predictions!

Application 4 — Speech Recognition

TASK: Audio → Text transcript

INPUT: Audio waveform

(sequence of amplitude values over time)

PROCESS:

1. Feature extraction: Audio → MFCC features

   (13-40 features per 25ms frame)

2. LSTM processes MFCC sequence

   Bidirectional LSTM (know full audio!)

   Stack of 4-6 LSTM layers!

3. CTC Loss (Connectionist Temporal Classification)

Special loss for alignment between

audio frames and text characters!

4. Decode: LSTM outputs → character probabilities

→ Best sequence of characters = transcript!

REAL EXAMPLES:

Google's speech recognition

Apple's Siri

Amazon's Alexa

All use deep LSTM (or now Transformers)!

PART 8 — Complete Comparison

RNN Family Complete Comparison

Feature	**Simple RNN**	**LSTM**	**GRU**	**Bidirectional**	**Stacked**
Gates	0	3	2	N/A (wrapper)	N/A
States	1	2	1	2×	L×
Long-term memory		Excellent	Good	✅	✅
Vanishing gradient	Severe	Solved	Solved	✅	✅
Training speed	Fastest	Slowest	Fast	2× slower	L× slower
Parameters	Fewest	Most	Medium	2×	L×
Context	Past only	Past only	Past only	Past + Future	Past only
Abstraction	Low	Low	Low	Low	High!

When to Use What

SIMPLE RNN:

→ Very short sequences (< 10 steps)

→ Simple patterns

→ Learning about RNNs

→ Almost never in production!

LSTM:

→ Very long sequences (100+ steps)

→ Complex language tasks

→ When GRU isn't enough

→ Speech recognition

→ Handwriting generation

GRU:

→ Most sequence tasks!

→ Start here: faster training

→ Similar performance to LSTM

→ Sentiment analysis

→ Time series forecasting

BIDIRECTIONAL LSTM/GRU:

→ Text classification (see whole sentence)

→ Named entity recognition

→ Machine translation encoder

→ When both context directions matter

→ NOT for real-time prediction!

STACKED LSTM/GRU:

→ Complex language understanding

→ When need hierarchical features

→ More data available

→ Better performance needed

→ 2-3 layers usually sufficient

ENCODER-DECODER:

→ Sequence to sequence tasks!

→ Machine translation

→ Text summarization

→ Question answering

→ Image captioning

Complete Workflow — Building RNN/LSTM/GRU

STEP 1: PROBLEM DEFINITION

→ What is the input sequence?

→ What is the output? (classification/regression/generation)

→ What type: many-to-one, many-to-many?

→ Sequence length?

STEP 2: DATA PREPARATION

→ Collect sequential data!

→ Preprocessing:

Text: tokenize → integer encoding → embedding

Audio: extract MFCC features

Time series: normalize, create windows

→ Train/validation/test split

→ Pad sequences to same length

→ Create data loaders with batching

STEP 3: MODEL ARCHITECTURE

→ Choose: Simple RNN, LSTM, or GRU?

→ Bidirectional? (usually yes for text classification!)

→ How many layers? (start with 1-2!)

→ Hidden size? (start with 64-256!)

→ Dropout between layers? (0.2-0.5)

→ Output layer? (softmax/linear based on task)

STEP 4: TRAINING SETUP

→ Optimizer: Adam (lr=0.001) or SGD (lr=0.01)

→ Loss: CrossEntropy (classification), MSE (regression)

→ Gradient clipping: max_norm=1.0 or 5.0 (always!)

→ Batch size: 32 or 64

→ Epochs: 50-200 with early stopping

STEP 5: TRAINING

→ Forward pass through LSTM/GRU

→ Compute loss

→ Backward pass (BPTT)

→ Clip gradients!

→ Update weights

→ Monitor train/val loss each epoch!

→ Early stopping with patience=10

STEP 6: EVALUATION

→ Test set performance

→ Classification: accuracy, F1, confusion matrix

→ Regression: RMSE, MAE, R^2

→ Qualitative: check some predictions!

STEP 7: IMPROVEMENTS

→ Underfit: more layers, larger hidden, more epochs

→ Overfit: dropout, less layers, more data

→ Slow: reduce hidden size, use GRU instead LSTM

→ NaN: reduce learning rate, add gradient clipping
